

On the Conditional Covering Problem on Paths

Chien-Hsin Lin and Biing-Feng Wang

Department of Computer Science, National Tsing Hua University

Hsinchu, Taiwan 30043, Republic of China

jsing@venus.cs.nthu.edu.tw, bfwang@cs.nthu.edu.tw

Abstract

Consider a graph G , in which the set of vertices represents the set of demand points as well as the set of potential facility sites. The classical covering location problem seeks to minimize the total location cost required to cover all vertices of G . The conditional covering problem (CCP) has the same objective but with an additional constraint: each selected facility site must be covered by at least one other selected site. The CCP arises in several military and security applications that are becoming increasingly important. It is strongly NP-hard. In several special cases, polynomial results are known. In this paper, an $O(n \log n)$ -time algorithm is presented for the CCP on a path. The presented algorithm improves the previous upper bound from $O(n^3)$.

Key Words: graph algorithms, covering location problems, conditional covering, dynamic programming

1. Introduction

Over three decades, covering location problems on networks have received much attention from researchers in the fields of transportation and communication. One of the classical problems is the *location set covering problem* (LSCP) [18]. Given a set of demand points and a set of potential facility sites, the problem is to determine the minimum number of facility sites so that each given demand point is covered by at least one facility. Due to practical considerations, numerous extensions of the LSCP have been defined and studied [17]. A useful extension is the *conditional covering problem* (CCP) [3, 12]. The CCP has the same objective as the LSCP but with an additional constraint: each selected facility site must be covered by at least one other selected site. It is easy to motivate the inter-facility support imposed by the CCP [12]. For example, consider the deployment of distributors that have non-overlapping service regions. The inter-facility support is useful in order to offset shortages of a distributor by a nearby distributor promptly.

The CCP was firstly introduced by Moon and Chaudhry [12]. They gave an integer linear programming model for the problem and then approached its solution by linear relaxation and cutting plane techniques. Efficient heuristics algorithms for the CCP had been extensively studied in [2, 3, 9, 11]. Moon and Papayanopoulos [13] studied the CCP on a tree, in which the set of vertices represents the set of demand points and the set of all points on the edges represents the set of potential facility sites. They had a linear-time algorithm for the CCP on a tree. Recently, Lunday, Smith, and Goldberg [10] introduced a modified version of the CCP. It is defined as follows. Let $G = (V, E)$ be a graph, where V is the vertex set and E is the edge set. The vertex set V represents the set of demand points as well as the set of potential facility sites. A facility located at a vertex $i \in V$ incurs a positive cost and provides a positive coverage radius. A facility located at a vertex covers all vertices within its coverage radius, except for the vertex at which it is located. The modified version of the CCP seeks to minimize the sum of facility location costs required to cover all vertices. There are two differences between the modified version and the original CCP. First, in the original CCP, each demand point is specified with a distance constraint and a demand point is covered by a facility only if the facility is located within its distance constraint. Second, in the original CCP, the set of demand points and the set of potential facility sites may be different. This modified version arises in several military and security applications that are becoming increasingly important. A motivating example is as follows [10]. Consider the location of Weapons of Mass Destruction Civil Support Teams in the United States. The teams are located in order to rapidly respond to biological or chemical terrorist incidents in major cities. In order to provide access to transportation hubs, the teams will be located in major cities. Hence, the set of feasible locations and the set of cities to be protected are the same. A team cannot cover the city at which it is located, since a biological or chemical attack to the city may render the team incapable of performing its mission. Therefore, every team must be covered by one other team.

Lunday, Smith, and Goldberg's modified

version is a natural variant of the CCP. Moon and Chaudhry's integer linear programming model as well as the heuristics algorithms in [2, 3, 9, 11] are applicable to their modified version. This modified version is the focus of this paper. For convenience, in the remainder of this paper, this modified version is simply referred as the CCP. The *total dominating set problem*, introduced by Cockayne, Dawes, and Hedetniemi [4], is a special case of the CCP, in which all edge lengths, coverage radii, and costs are 1. The total dominating set problem is strongly NP-hard [14]. Therefore, the CCP is also strongly NP-hard. In several special cases, polynomial-time algorithms are known for the total dominating set problem. Let $n = |V|$. Pfaff, Laskar, and Hedetniemi [15] had an $O(n)$ -time algorithm for series-parallel graphs, Laskar *et al.* [7] had an $O(n)$ -time algorithm for trees, and Bertossi and Gori [1] had an $O(n \log n)$ -time algorithm for interval graphs. Lunday, Smith, and Goldberg [10] studied the CCP on a path, motivated by some military applications. They gave an $O(n^2)$ -time algorithm for the special case that all coverage radii are uniform, and an $O(n)$ -time algorithm for the special case that all coverage radii and all location costs are uniform. Later, Horne and Smith [5] presented an $O(n^2)$ -time algorithm for the CCP on a path, with non-uniform coverage radii and location costs. In the same paper, using their path algorithm as a procedure, they also derived an $O(n^2)$ -time algorithm for the CCP on an extended star, which is a rooted tree consisting of one or more paths, all incident to the root. Unfortunately, there is a flaw in their path algorithm [8]. (See Appendix 1.) For the CCP on a tree, Horne and Smith [6] had an $O(n^4)$ -time algorithm. When applying their tree algorithm to paths, the time complexity reduces to $O(n^3)$. In this paper, an $O(n \log n)$ -time algorithm is presented for the CCP on a path.

The remainder of this paper is organized as follows. In Section 2, notation and preliminaries are given. In Section 3, an $O(n^2)$ -time dynamic programming algorithm is presented. The presented algorithm is a revision of the algorithm in [5]. Then, in Section 4, we show how to modify the algorithm in Section 3 so as to solve the CCP problem on a path in $O(n \log n)$ time. Finally, in Section 5, we conclude this paper.

2. Notation and preliminaries

Let $G = (V, E)$ be a path graph, where the vertex-set is $V = \{1, 2, \dots, n\}$ and the edge-set is $E = \{(1, 2), (2, 3), \dots, (n-1, n)\}$. Each edge of G has a nonnegative length. For any $i, j \in V$, let $d(i, j)$ be the distance between a and b along the path. A

facility located at a vertex $i \in V$ incurs a positive cost of c_i and provides a positive coverage radius of R_i . For any subset $S \subseteq V$, define the *cost* of S to be $c(S) = \sum_{i \in S} c_i$. A vertex $i \in V$ is *covered* by a vertex $j \in V$ if $i \neq j$ and $d(i, j) \leq R_j$, and is *covered* by a subset $S \subseteq V$ if it is covered by some vertex in S . A subset $S \subseteq V$ is called a *cover* of V if it covers all $i \in V$. The *conditional covering problem* is to find a cover of V with the minimum cost.

For each $i \in V$, the *lower reach* of i is $h(i) = \min\{j \mid j \in V, d(i, j) \leq R_i\}$, and the *upper reach* of i is $g(i) = \max\{j \mid j \in V, d(i, j) \leq R_i\}$. A vertex $i \in V$ *dominates* another vertex $j \in V$ if $[h(j), g(j)] \subset [h(i), g(i)]$. For each $i \in V$, let $D(i)$ be the set of vertices dominated by i . Let U be the sequence obtained by ordering the vertices in V by the following rule: a vertex i appears before another vertex j if and only if (1) $g(i) < g(j)$, (2) $g(i) = g(j)$ and $h(i) > h(j)$, or (3) $g(i) = g(j)$, $h(i) = h(j)$, and $i < j$. For each $i \in V$, define the *rank* of i , denoted by $r(i)$, to be the position of i in U . Then, we have the following.

Lemma 1. For each $j \in D(i)$, $r(j) < r(i)$.

Proof: Since $j \in D(i)$, either $h(j) \geq h(i)$ and $g(j) < g(i)$, or $h(j) > h(i)$ and $g(j) \leq g(i)$. In both cases, j appears before i in the sequence U . Thus, the lemma holds. $\square$

Lemma 2. For any $i, j \in V$, if $r(j) < r(i)$ and $j \notin D(i)$, we have $j < i$.

Proof: Assume that $j > i$. Then, if $[h(j), g(j)] \subset [h(i), g(i)]$, $j \in D(i)$; otherwise, since $j > i$, it is easy to see that $r(j) > r(i)$. Thus, $r(j) < r(i)$ and $j \notin D(i)$ only if $j < i$. Therefore, the lemma holds. $\square$

3. An $O(n^2)$ -time algorithm

Our algorithm is based on the dynamic programming approach. Let $i \in V$ be a vertex. An *i -cover* is a subset of V that covers vertex 1 through vertex i . A *protected i -placement* is an i -cover S such that $i \in S$ and all vertices in S are with ranks $\leq r(i)$. An *optimal protected i -placement* is a protected i -placement of the minimum cost. An *unprotected i -placement* is an $(i-1)$ -cover S such that $i \in S$ and all vertices in S are with ranks $\leq r(i)$. An *optimal unprotected i -placement* is an unprotected i -placement of the minimum cost. Define the *protected cost* $p(i)$ to be the cost of an optimal protected i -placement. If no such placement exists, $p(i) = \infty$. Similarly, define the *unprotected cost* $u(i)$ to be the cost of an optimal unprotected i -placement. If no such placement exists, $u(i) = \infty$. For convenience, we assume that there is a dummy vertex $n+1$ with

$d(n, n + 1) = \infty$ and $R_{n+1} = c_{n+1} = 0$. Then, our problem becomes to find an optimal unprotected $(n + 1)$ -placement. Sections 3.1 and 3.2 give recurrences of $p(i)$ and $u(i)$, respectively.

3.1. Protected cost

Let $\alpha(i)$ be the vertex with the minimum cost among all vertices in $D(i)$ that cover i . If no such vertex exists, let $\alpha(i) = -1$ and $c_{\alpha(i)} = \infty$. The vertex $\alpha(i)$ is called the *guarding vertex* of i . We have the following.

Lemma 3. If $p(i) \neq \infty$, there exists an optimal protected i -placement that does not contain any vertices in $D(i) - \{\alpha(i)\}$.

Proof: Consider an optimal protected i -placement S . Let $X = S \cap D(i)$. If X does not cover i , we remove X from S ; otherwise, we replace X with $\alpha(i)$. Note that by Lemma 1, if $\alpha(i)$ exists, $r(\alpha(i)) < r(i)$. Since i dominates all vertices in X , after the removal or replacement, S is still a protected i -placement. The removal or replacement does not increase the cost. Thus, the lemma holds. $\square$

Assume that $p(i) \neq \infty$. Let $P(i)$ be an optimal protected i -placement that does not contain any vertices in $D(i) - \{\alpha(i)\}$. Let $P'(i) = P(i) - \{i, \alpha(i)\}$. For convenience, we assume that there is a dummy vertex 0 with $d(0, 1) = \infty$ and $R_0 = c_0 = 0$. The protected and unprotected cost of vertex 0 is $p(0) = u(0) = 0$. We assume that vertex 0 is included in $P(i)$, so that $P'(i)$ is nonempty. Let k be the vertex with the largest rank in $P'(i)$. Note that $k < i$ by Lemma 2. Since k is the vertex with the largest rank in $P'(i)$, no vertex in $P'(i)$ has an upper reach larger than $g(k)$. Thus, $P'(i)$ covers i if and only if k covers i , from which we conclude that $\alpha(i) \in P(i)$ if k does not cover i . On the other hand, if k covers i , $\alpha(i) \notin P(i)$; otherwise, we can remove $\alpha(i)$ to reduce the cost. Therefore, we have the following.

Lemma 4. k covers i if and only if $\alpha(i) \notin P(i)$.

Lemma 5. $\{k, i\}$ covers vertices $k + 1, k + 2, \dots, i - 1$.

Proof: No vertex in $P'(i)$ has an upper reach larger than $g(k)$. Since $\alpha(i) \in D(i)$, the lower reach of $\alpha(i)$ is not smaller than $h(i)$. Thus, if a vertex between k and i is not covered by $\{k, i\}$, it is also not covered by $P(i)$. Since $P(i)$ covers all vertices between k and i , the lemma holds. $\square$

Lemma 6. $P'(i)$ covers vertices $1, 2, \dots, k - 1$.

Proof: Since $k < i$ and $k \notin D(i)$, we have $h(k) \leq$

$h(i)$. Thus, $\{i, \alpha(i)\}$ does not cover vertices $1, 2, \dots, h(k) - 1$, from which we conclude that $P'(i)$ covers $1, 2, \dots, h(k) - 1$. Moreover, since $k \in P'(i)$, $P'(i)$ also covers $h(k), h(k) + 1, \dots, k - 1$. Therefore, the lemma holds. $\square$

The following lemma describes the optimal substructure of $P(i)$. Due to the page limit, its proof is omitted. (See appendix 2.)

Lemma 7. If k is not covered by i , $P'(i)$ is an optimal protected k -placement; otherwise, $P'(i)$ is an optimal unprotected k -placement.

The set $\{k \mid k \in V, r(k) < r(i), k \notin D(i)\}$ contains all possible candidates of k . We partition it into the following subsets:

- $K_1(i) = \{k \mid k \in V, r(k) < r(i), k \notin D(i), g(k) \geq i, k \geq h(i)\}$, which contains the vertices that cover i and are covered by i ;
- $K_2(i) = \{k \mid k \in V, r(k) < r(i), k \notin D(i), g(k) < i, k \geq h(i)\}$, which contains the vertices that do not cover i , but are covered by i ;
- $K_3(i) = \{k \mid k \in V, r(k) < r(i), k \notin D(i), g(k) \geq i, k < h(i)\}$, which contains the vertices that cover i , but are not covered by i ;
- $K_4(i) = \{k \mid k \in V, r(k) < r(i), k \notin D(i), g(k) < i, k < h(i), g(k) + 1 \geq h(i)\}$, which contains the vertices k such that k and i do not cover each other, but $\{k, i\}$ covers all vertices between k and i ; and
- $K_5(i) = \{k \mid k \in V, r(k) < r(i), k \notin D(i), i > g(k), k < h(i), g(k) + 1 < h(i)\}$, which contains the vertices k such that k and i do not cover each other, and $\{k, i\}$ does not cover all vertices between k and i .

Now, based on Lemmas 4, 5, and 7, we proceed to derive a recurrence of $p(i)$. The following cases are discussed.

- Case 1: $k \in K_1(i)$. In this case, $\alpha(i) \notin P(i)$ and $P'(i)$ is an optimal unprotected k -placement. Thus, $p(i) = u(k) + c_i$.
- Case 2: $k \in K_2(i)$. In this case, $\alpha(i) \in P(i)$ and $P'(i)$ is an optimal unprotected k -placement. Thus, $p(i) = u(k) + c_i + c_{\alpha(i)}$.
- Case 3: $k \in K_3(i)$. In this case, $\alpha(i) \notin P(i)$ and $P'(i)$ is an optimal protected k -placement. Thus, $p(i) = p(k) + c_i$.
- Case 4: $k \in K_4(i)$. In this case, $\alpha(i) \in P(i)$ and $P'(i)$ is an optimal protected k -placement. Thus, $p(i) = p(k) + c_i + c_{\alpha(i)}$.
- Case 5: $k \in K_5(i)$. By Lemma 5, this case does not exist.
- Case 6: k does not exist. In this case, there does not exist any protected i -placement. Thus, $p(i) = \infty$.

For convenience, throughout this paper, we assume that the minimum of an empty set is ∞ . Based on the above discussion, we obtain the following: for each $i \geq 1$,

$$(R1) \quad p(i) = \min\left\{\min_{k \in K_1(i)} u(k), \min_{k \in K_2(i)} u(k) + c_{\alpha(i)}, \min_{k \in K_3(i)} p(k), \min_{k \in K_4(i)} p(k) + c_{\alpha(i)}\right\} + c_i.$$

3.2. Unprotected cost

Assume that $u(i) \neq \infty$. Let $U(i)$ be an optimal unprotected i -placement. Let $U'(i) = U(i) - \{i\}$. Since i dominates all vertices in $D(i)$, $U(i)$ does not contain any vertices in $D(i)$, and thus by Lemma 2 for any vertex $j \in U'(i)$, $j < i$. Let k be the vertex with the largest rank in $U'(i)$. Then, similar to Lemma 7, we can prove the following.

Lemma 8. If k is not covered by i , $U'(i)$ is an optimal protected k -placement; otherwise, $U'(i)$ is an optimal unprotected k -placement.

Based on Lemma 8, we obtain the following: for each $i \geq 1$,

$$(R2) \quad u(i) = \min\left\{\min_{k \in K_1(i) \cup K_2(i)} u(k), \min_{k \in K_3(i) \cup K_4(i)} p(k)\right\} + c_i.$$

As mentioned, our problem is to find an optimal unprotected $(n + 1)$ -placement. Based on recurrences (R1) and (R2), it is easy to compute the value of $u(n + 1)$ and then determine an optimal unprotected $(n + 1)$ -placement in $O(n^2)$ time. We have the following.

Theorem 1. The conditional covering problem on a path can be solved in $O(n^2)$ time.

Figure 1 gives an example of $n = 5$. For the vertex 6, we have $K_1(6) = K_2(6) = K_3(6) = \emptyset$ and

$K_4(6) = \{4, 5\}$. Thus, $p(6) = \min\{p(4), p(5)\} + c_6 + c_{\alpha(6)} = \infty$, and $u(6) = \min\{p(4), p(5)\} + c_6 = 4$. In this example, $\{3, 4\}$ is an optimal unprotected 6-placement.

4. An improved algorithm

In this section, we show how to modify the algorithm in Section 3, so as to solve the conditional covering problem on a path in $O(n \log n)$ time. For ease of discussion, we define a directed acyclic graph H corresponding to recurrences (R1) and (R2) as follows:

- (1) the vertex set is $V(H) = \{s = u_0 = p_0, p_1, u_1, p_2, u_2, \dots, p_{n+1}, u_{n+1}\}$;
- (2) the edge set is $E(H) = E_1 \cup E_2 \cup E_3 \cup E_4 \cup E_5 \cup E_6$, where $E_1 = \{(u_k, p_i) \mid 1 \leq i \leq n+1, k \in K_1(i)\}$, $E_2 = \{(u_k, p_i) \mid 1 \leq i \leq n+1, k \in K_2(i)\}$, $E_3 = \{(p_k, p_i) \mid 1 \leq i \leq n+1, k \in K_3(i)\}$, $E_4 = \{(p_k, p_i) \mid 1 \leq i \leq n+1, k \in K_4(i)\}$, $E_5 = \{(u_k, u_i) \mid 1 \leq i \leq n+1, k \in K_1(i) \cup K_2(i)\}$, and $E_6 = \{(p_k, u_i) \mid 1 \leq i \leq n+1, k \in K_3(i) \cup K_4(i)\}$; and
- (3) all edges $(u_k, p_i) \in E_1$, $(p_k, p_i) \in E_3$, $(u_k, u_i) \in E_5$, $(p_k, u_i) \in E_6$ have a length of c_i , and all edges $(u_k, p_i) \in E_2$ and $(p_k, p_i) \in E_4$ have a length of $c_i + c_{\alpha(i)}$.

Figure 2 depicts the graph H for the example in Figure 1. For clarity, edges of length ∞ are not depicted. For each vertex $x \in V(H)$, let $l(x)$ be the length of a shortest path from s to x . Then, we have $l(p_i) = p(i)$ and $l(u_i) = u(i)$ for each $i \in V$. Let $e \in E(H)$ be an edge directed to p_i or u_i . If the length of e is c_i , we define $\pi(e) = \{i\}$ and call e a *normal edge* of i ; otherwise, we define $\pi(e) = \{i, \alpha(i)\}$ and call e a *guarding edge* of i . Note that e is a guarding edge only if its tail is p_i . For each path X from s to a vertex x in H , let $\pi(X) = \cup_{e \in X} \pi(e)$ and $l(X)$ be its length. For example, in Figure 2, (s, p_3) is a guarding edge with $\pi((s, p_3)) = \{3, \alpha(3)\}$

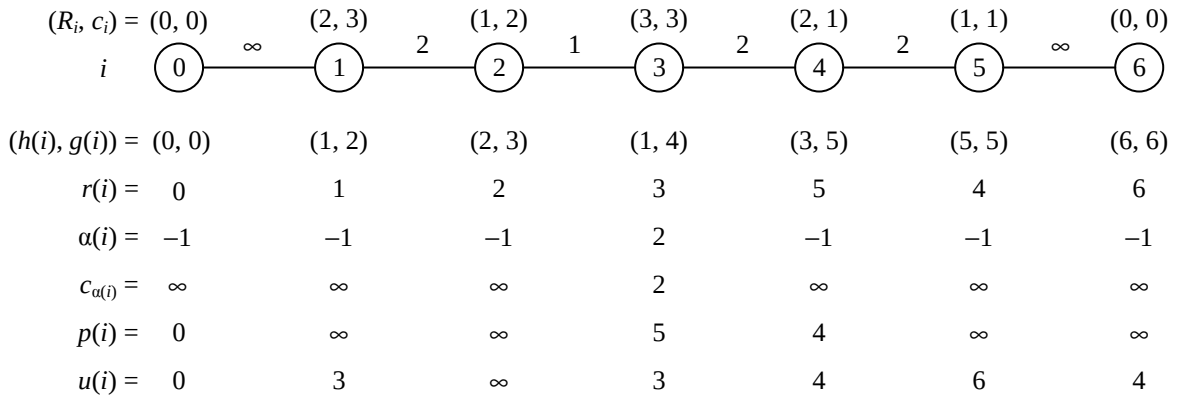
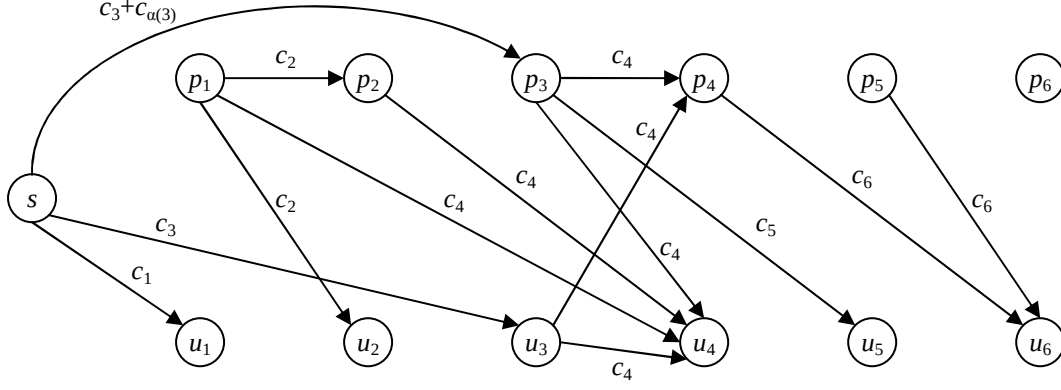


Figure 1. An example.


 Figure 2. The graph H .

$\{2, 3\}$, (p_3, p_4) is a normal edge with $\pi((p_3, p_4)) = \{4\}$, and the path $X = (s, p_3, p_4)$ is with $\pi(X) = \{2, 3, 4\}$. According to the definitions of a protected i -placement and an unprotected i -placement, the following lemma can be easily obtained by an induction proof.

Lemma 9. For any path X from s to a vertex p_i , $\pi(X)$ is a protected i -placement with cost $l(X)$; and for any path X from s to a vertex u_i , $\pi(X)$ is an unprotected i -placement with cost $l(X)$.

The following is a corollary of Lemma 9.

Lemma 10. For any path X from s to a vertex p_i , $\pi(X)$ is an i -cover that includes i and has cost $\leq l(X)$; and for any path X from s to a vertex u_i , $\pi(X)$ is an $(i - 1)$ -cover that includes i and has cost $\leq l(X)$.

Let $x \in V(H)$ be a vertex. Let $A(x) = \{y \mid (y, x) \in E(H) \text{ is a normal edge}\}$ and $B(x) = \{y \mid (y, x) \in E(H) \text{ is a guarding edge}\}$. Then, recurrences (R1) and (R2) can be equivalently expressed as follows: for each vertex $x = p_i$ or $x = u_i$,

$$(R3) \quad l(x) = \min\{\min_{y \in A(x)} l(y), \min_{y \in B(x)} l(y) + c_{\alpha(i)}\} + c_i.$$

Iteratively, the value of $l(x)$ can be computed for each $x \in V(H)$ according to (R3). There are two difficulties in the computation. First, we need to find $c_{\alpha(i)}$ for every $i \in V$. We overcome this difficulty by modifying the length of each guarding edge to avoid the finding. The modification may decrease the value of $l(x)$ for each $x \neq u_{n+1}$, but it does not change the value of $l(u_{n+1})$. Second, we need to determine $\min_{y \in A(x)} l(y)$ and $\min_{y \in B(x)} l(y)$. According to (R1) and (R2), it is difficult to do the determination efficiently. To overcome this difficulty, our idea is to modify the graph H by adding additional edges so that $A(x)$ and $B(x)$ becomes more "regular" and thus the

determination can be done efficiently.

We proceed to show how to avoid the finding of $c_{\alpha(i)}$. Consider a fixed $i \in V$. Let $\beta(i)$ be the vertex with the minimum cost in V that covers i . If no such vertex exists, $\beta(i) = -1$ and $c_{\beta(i)} = \infty$. Let $e \in E(H)$ be a guarding edge directed to p_i . By definition, the length of e is $c_i + \alpha(i)$ and $\pi(e) = \{i, \alpha(i)\}$. Let X be an arbitrary path from s to p_i in which e is the last edge. Let X' be the sub-path $X - e$. By Lemma 10, $\pi(X) = \pi(X') \cup \{i, \alpha(i)\}$ is an i -cover with cost $\leq l(X') + c_i + c_{\alpha(i)}$. Since $\alpha(i) \in D(i)$, $\pi(X') \cup \{i\}$ covers $\{1, 2, \dots, i - 1\}$. Thus, $\pi(X') \cup \{i, \beta(i)\}$ is an i -cover with cost $\leq l(X') + c_i + c_{\beta(i)}$. And thus, if we change the length of e into $c_i + c_{\beta(i)}$ and $\pi(e)$ into $\{i, \beta(i)\}$, Lemma 10 still holds. By repeating the same argument, it follows that doing such changes for all guarding edges does not invalidate Lemma 10. Therefore, we modify (R1) and (R2) into the following: for each $i \geq 1$,

$$(R1') \quad p_1(i) = \min\{\min_{k \in K_1(i)} u_1(k), \min_{k \in K_2(i)} u_1(k) + c_{\beta(i)}, \min_{k \in K_3(i)} p_1(k), \min_{k \in K_4(i)} p_1(k) + c_{\beta(i)}\} + c_i, \text{ and}$$

$$(R2') \quad u_1(i) = \min\{\min_{k \in K_1(i) \cup K_2(i)} u_1(k), \min_{k \in K_3(i) \cup K_4(i)} p_1(k)\} + c_i.$$

Let H_1 be the graph corresponding to recurrences (R1') and (R2'). The graph H_1 is just the graph obtained from H by changing the length of e into $c_i + c_{\beta(i)}$ and $\pi(e)$ into $\{i, \beta(i)\}$ for each guarding edge e of every $i \in V$. Therefore, we have the following.

Lemma 11. Lemma 10 holds as well for the graph H_1 .

Figure 3 shows the graph H_1 for the example in Figure 1. In H , the length of a path X from s to any vertex is equal to the cost of $\pi(X)$. However, this is not true for H_1 , since the cost of a vertex $i \in V$ may

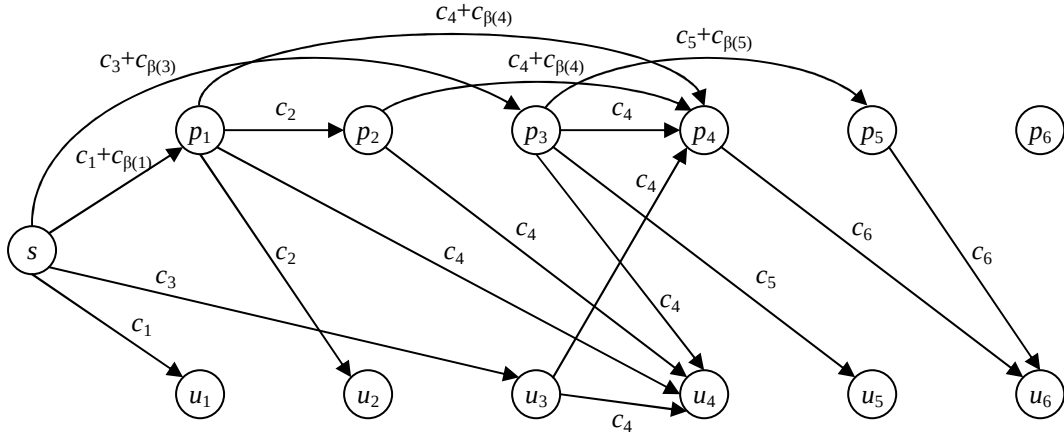


Figure 3. The graph H_1 , where $(\beta(1), \beta(2), \dots, \beta(6)) = (3, 1, 4, 3, 4, -1)$.

be counted more than once in the length of a path. For example, for the path $X = (s, p_3, p_4, u_6)$ in Figure 3, its length is $c_3 + c_4 + c_4 + c_6 = 5$, but its cost is $c_3 + c_4 + c_6 = 4$. For each $x \in V(H_1)$, let $l_1(x)$ be the length of a shortest path from s to x . Clearly, $l_1(x) \leq l(x)$ for any $x \in V(H)$. However, $l_1(u_{n+1}) = l(u_{n+1})$, since every path from s to u_{n+1} in H_1 still represents an n -cover.

By using the plane-sweep technique [16], the following is obtained. (See Appendix 2.)

Lemma 12. The value of $c_{\beta(i)}$ can be computed in $O(n \log n)$ time for every $i \in V$.

Remark 1. With some efforts, the value of $c_{\alpha(i)}$ can also be computed in $O(n \log n)$ time for every $i \in V$. However, the computation is more complicated.

Next, we show how to further modify H_1 so that $\min_{y \in A(x)} l_1(y)$ and $\min_{y \in B(x)} l_1(y)$ can be determined efficiently for each $x \in V(H)$. By Lemma 2, for any $k \in V$, if $r(k) < r(i)$ and $k \notin D(i)$, we have $k < i$. By relaxing the constraint that $r(k) < r(i)$ and $k \notin D(i)$ by the constraint that $k < i$, we obtain from $K_1(i)$, $K_2(i)$, $K_3(i)$, and $K_4(i)$, respectively, the following subsets: $K'_1(i) = \{k \mid k \in V, k < i, g(k) \geq i, k \geq h(i)\}$, $K'_2(i) = \{k \mid k \in V, k < i, g(k) < i, k \geq h(i)\}$, $K'_3(i) = \{k \mid k \in V, k < i, g(k) \geq i, k < h(i)\}$, and $K'_4(i) = \{k \mid k \in V, k < i, g(k) < i, k < h(i), g(k) + 1 \geq h(i)\}$. Then, by replacing K_1, K_2, K_3, K_4 with K'_1, K'_2, K'_3, K'_4 , respectively, in (R1') and (R2'), we obtain the following: for each $i \geq 1$,

$$(R1'') \quad p_2(i) = \min\left\{ \min_{k \in K'_1(i)} u_2(k), \min_{k \in K'_2(i)} u_2(k) + c_{\beta(i)}, \min_{k \in K'_3(i)} p_2(k), \min_{k \in K'_4(i)} p_2(k) + c_{\beta(i)} \right\} + c_i, \text{ and}$$

$$(R2'') \quad u_2(i) = \min\left\{ \min_{k \in K'_1(i) \cup K'_2(i)} u_2(k), \min_{k \in K'_3(i) \cup K'_4(i)} p_2(k) \right\} + c_i.$$

Let H_2 be the graph corresponding to recurrences

(R1'') and (R2''). Then, H_2 is the graph obtained from H_1 by adding edges as follows: for each $i \geq 1$,

- (1) add two normal edges (u_k, p_i) , (u_k, u_i) for every $k \in K'_1(i) - K_1(i)$,
- (2) add a guarding edge (u_k, p_i) and a normal edge (u_k, u_i) for every $k \in K'_2(i) - K_2(i)$,
- (3) add two normal edges (p_k, p_i) , (p_k, u_i) for every $k \in K'_3(i) - K_3(i)$, and
- (4) add a guarding edge (p_k, p_i) and a normal edge (p_k, u_i) for every $k \in K'_4(i) - K_4(i)$.

Lemma 13. Lemma 10 holds as well for the graph H_2 .

Proof: Consider the adding of edges in (1). Let u_k be a vertex with $k \in K'_1(i) - K_1(i)$. Let X be any path from s to u_k in H_1 . By Lemma 10, $\pi(X)$ is a $(k-1)$ -cover that includes k and has cost $\leq l_1(X)$. Since i and k cover each other, $\pi(X) \cup \{i\}$ is an i -cover (as well as an $(i-1)$ -cover) that includes i and has cost $\leq l_1(X) + c_i$. Thus, we can add two normal edges (u_k, p_i) , (u_k, u_i) in H_1 without invalidating Lemma 10. By repeating the same argument, we conclude that after the adding of all the edges in (1), Lemma 10 still holds. Similarly, the adding of edges in (2), (3), and (4) does not invalidate Lemma 10. Therefore, the lemma holds.

For each $x \in V(H_2)$, let $l_2(x)$ be the length of a shortest path from s to x . Since Lemma 10 holds for H_2 , $l_2(u_{n+1}) = l_1(u_{n+1}) = l(u_{n+1})$. We have the following.

Theorem 2. $u_2(n+1)$ is the minimum cost of a cover of V .

The constraints on which a vertex $k \in V$ is belonging to $K'_1(i)$, $K'_2(i)$, $K'_3(i)$, and $K'_4(i)$ are much simpler than those of $K_1(i)$, $K_2(i)$, $K_3(i)$, and $K_4(i)$. However, a few more modifications are required. Define the following subsets: (See

Figure 4 for an illustration.)

$K'_5(i) = \{k \in V, k < i, k \geq h(i)\}$, which is $K'_1(i) \cup K'_2(i)$;
 $K'_6(i) = \{k \in V, k < i, g(k) \geq i\}$, which is $K'_1(i) \cup K'_3(i)$;
 $K'_7(i) = \{k \in V, k < i, h(i) - 1 \leq g(k) < i, k \geq h(i)\}$, which is a subset of $K'_2(i)$; and
 $K'_8(i) = \{k \in V, k < i, h(i) - 1 \leq g(k)\}$, which is $K'_1(i) \cup K'_3(i) \cup K'_4(i) \cup K'_7(i)$.

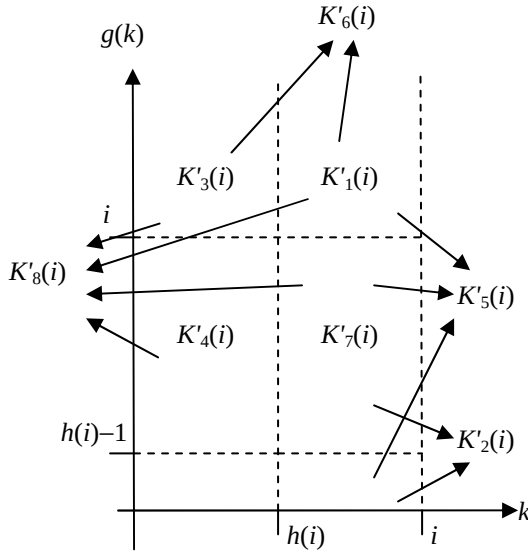


Figure 4. $K'_1(i), K'_2(i), \dots, K'_8(i)$.

Consider the recurrence (R1''). Let $M_1 = \min_{k \in K'_1(i)} u_2(k)$, $M_2 = \min_{k \in K'_2(i)} u_2(k) + c_{\beta(i)}$, $M_3 = \min_{k \in K'_3(i)} p_2(k)$, and $M_4 = \min_{k \in K'_4(i)} p_2(k) + c_{\beta(i)}$. Then, $p_2(i) = \min\{M_1, M_2, M_3, M_4\} + c_i$. Since $M_1 = \min_{k \in K'_1(i)} u_2(k) \leq \min_{k \in K'_1(i)} u_2(k) + c_{\beta(i)}$, we can modify M_2 into $M'_2 = \min_{k \in K'_1(i) \cup K'_2(i)} u_2(k) + c_{\beta(i)} = \min_{k \in K'_5(i)} u_2(k) + c_{\beta(i)}$ without changing the value of $p_2(i)$. By comparing (R1'') and (R2''), it is easy to observe that $p_2(i) \geq u_2(i)$ for any $i \in V$. Thus, $M_1 \leq \min_{k \in K'_1(i)} p_2(k)$. And thus, we can modify M_3 into $M'_3 = \min_{k \in K'_1(i) \cup K'_3(i)} p_2(k) = \min_{k \in K'_6(i)} p_2(k)$. Since $K'_7(i) \subseteq K'_2(i)$ and $p_2(k) \geq u_2(k)$, we have $M_2 \leq \min_{k \in K'_7(i)} p_2(k) + c_{\beta(i)}$. Moreover, we have $M'_3 = \min_{k \in K'_6(i)} p_2(k) \leq \min_{k \in K'_6(i)} p_2(k) + c_{\beta(i)}$. Therefore, we can modify M_4 into $M'_4 = \min_{k \in K'_4(i) \cup K'_6(i) \cup K'_7(i)} p_2(k) + c_{\beta(i)} = \min_{k \in K'_8(i)} p_2(k) + c_{\beta(i)}$. Consequently, we can rewrite (R1'') as follows: for each $i \geq 1$,

$$(R1''') \quad p_2(i) = \min\left\{\min_{k \in K'_1(i)} u_2(k), \min_{k \in K'_5(i)} u_2(k) + c_{\beta(i)}, \min_{k \in K'_6(i)} p_2(k), \min_{k \in K'_8(i)} p_2(k) + c_{\beta(i)}\right\} + c_i.$$

Similarly, we can rewrite (R2'') as follows: for each $i \geq 1$,

$$(R2''') \quad u_2(i) = \min\left\{\min_{k \in K'_5(i)} u_2(k), \min_{k \in K'_8(i)} p_2(k)\right\} + c_i.$$

Now, we are ready to present an improved algorithm for the conditional covering problem on a path. In the course of the algorithm, three segment trees [16] T_1, T_2, T_3 , are maintained. For ease of presentation, define the following three operations:

BuildTree(S): build a segment tree T on a given set $S = \{(i, x_i, y_i) \mid i \in V\}$, where x_i and y_i are, respectively, the *key* and *value* of vertex i ;

RangeMin(T, a, b): return the smallest value in $\{y_i \mid a \leq x_i \leq b, i \in V\}$; and

Update(T, i, y): change the value of y_i into y .

Our algorithm is as follows.

Algorithm 1.

CONDITIONAL_COVERING_PATH

Input: a path graph $G = (V, E)$

Output: the minimum cost of a cover of V

begin

- 1 $V \leftarrow V \cup \{0, n+1\}$
- 2 $(c_0, R_0, d(0, 1), p_2(0), u_2(0)) \leftarrow (0, 0, \infty, 0, 0)$;
 $(c_{n+1}, R_{n+1}, d(n, (n+1))) \leftarrow (0, 0, \infty)$
- 3 compute $h(i), g(i)$, and $c_{\beta(i)}$ for each $i \in V$
- 4 $T_1 \leftarrow \text{BuildTree}(\{(i, i, \infty) \mid i \in V\})$
- 5 $T_2 \leftarrow \text{BuildTree}(\{(i, i, \infty) \mid i \in V\})$;
 Update($T_2, 0, u_2(0)$);
- 6 $T_3 \leftarrow \text{BuildTree}(\{(i, g(i), \infty) \mid i \in V\})$;
 Update($T_3, 0, p_2(0)$)
- 7 **for** $i \leftarrow 1$ **to** $n+1$ **do**
- 8 **begin**
- 9 $m_1 \leftarrow \text{RangeMin}(T_1, h(i), n)$
 /** $m_1 = \min_{k \in K'_1(i)} u_2(k)$
- 10 $m_2 \leftarrow \text{RangeMin}(T_2, h(i), n)$
 /** $m_2 = \min_{k \in K'_5(i)} u_2(k)$
- 11 $m_3 \leftarrow \text{RangeMin}(T_3, i, n)$
 /** $m_3 = \min_{k \in K'_6(i)} p_2(k)$
- 12 $m_4 \leftarrow \text{RangeMin}(T_3, h(i) - 1, n)$
 /** $m_4 = \min_{k \in K'_8(i)} p_2(k)$
- 13 $p_2(i) \leftarrow \min\{m_1, m_2 + c_{\beta(i)}, m_3, m_4 + c_{\beta(i)}\} + c_i$
- 14 $u_2(i) \leftarrow \min\{m_2, m_4\} + c_i$
- 15 Update($T_1, i, u_2(i)$) /** insert $u_2(i)$
- 16 Update($T_2, i, u_2(i)$) /** insert $u_2(i)$
- 17 Update($T_3, i, p_2(i)$) /** insert $p_2(i)$
- 18 **for** each k with $g(k) = i$ **do** Update(T_1, k, ∞)
 /** delete all $u_2(k)$ with $g(k) = i$

```

19 end
20 return ( $u_2(n + 1)$ )
end

```

In the course of the algorithm, the following invariant is maintained: at the beginning of the i th iteration of the for-loop in Lines 7-19, the sets of values, excluding ∞ , stored in T_1 , T_2 , and T_3 are, respectively, $\{u_2(k) \mid k \in V, k < i, g(k) \geq i\}$, $\{u_2(k) \mid k \in V, k < i\}$, and $\{p_2(k) \mid k \in V, k < i\}$. When $i = 1$, $\{u_2(k) \mid k \in V, k < i, g(k) \geq i\} = \emptyset$, $\{u_2(k) \mid k \in V, k < i\} = \{u_2(0)\}$, and $\{p_2(k) \mid k \in V, k < i\} = \{p_2(0)\}$. Thus, according to Lines 4-6, the invariant is true for $i = 1$. At the i th iteration, $1 \leq i \leq n$, we maintain the invariant for the next iteration as follows. First, we insert $u_2(i)$ into T_1 and T_2 , and insert $p_2(i)$ into T_3 . The insertions are done in Lines 15-17 by performing $\text{Update}(T_1, i, u_2(i))$, $\text{Update}(T_2, i, u_2(i))$, and $\text{Update}(T_3, i, p_2(i))$. Clearly, after the insertions, the invariant of T_2 and T_3 holds for $i + 1$. To maintain the invariant of T_1 for $i + 1$, we need to further remove all $u_2(k)$ with $g(k) = i$ in it. Therefore, for each such $u_2(k)$, we perform $\text{Update}(T_1, k, \infty)$ in Line 18 to remove it.

The correctness of Algorithm 1 is ensured by (R1''), (R2''), and the above invariant. The time complexity is analyzed as follows. Lines 1-2 take $O(1)$ time. By sorting, all $h(i)$ and $g(i)$ can be determined in $O(n \log n)$ time. By Lemma 12, all $c_{\beta(i)}$ can be computed in $O(n \log n)$ time. Thus, Line 3 takes $O(n \log n)$ time. Each BuildTree operation needs $O(n \log n)$ time, and each Update and RangeMin operation needs $O(\log n)$ time [16]. Thus, Lines 4-6 require $O(n \log n)$ time. Consider the for-loop in Lines 7-19. At the i th iteration, we need to perform four RangeMin operations and $q_i + 3$ Update operations, where q_i is the number of Update operations performed in Line 18. Each Update operation in Line 18 corresponds to the deletion of an $u_2(k)$ from T_1 . Each $u_2(k)$ may be deleted from T_1 at most once. Thus, $(q_1 + q_2 + \dots + q_{n+1}) = n$. And therefore, the for-loop requires $O(n \log n)$ time in total. Consequently, the time complexity of Algorithm 1 is $O(n \log n)$. The output of Algorithm 1 is the minimum cost of a cover of V . With slight modifications, Algorithm 1 can output a minimum cover of V as well. Therefore, we have the following.

Theorem 3. The conditional covering problem on a path can be solved in $O(n \log n)$ time.

5. Concluding Remarks

During the last decades, the CCP has attracted significant research efforts. In this paper, an $O(n \log n)$ -time algorithm was presented for the

CCP on a path. The presented algorithm improves the previous upper bound from $O(n^3)$. By using our path algorithm as a procedure, Horne and Smith's extended star algorithm in [5] can correctly work in $O(n^2)$ time. For the CCP on trees, the upper bound in [6] is $O(n^4)$. In [8], these two upper bounds are improved into $O(n^{3/2})$ and $O(n^3)$, respectively. One direction for further study is to design more efficient algorithms for extended stars and trees. Another direction is to design polynomial-time algorithms for more complex graph structures, such as interval graphs and serial-parallel graphs.

References

- [1] A. A. Bertossi and A. Gori, "Total domination and irredundance in weighted interval graphs," *SIAM Journal on Discrete Mathematics*, vol. 1, no. 3, pp. 317-327, 1988.
- [2] S. S. Chaudhry, "New heuristics for the conditional covering problem," *Opsearch*, vol. 30, no. 1, pp. 42-47, 1993.
- [3] S. S. Chaudhry, I. D. Moon, and S. T. McCormick, "Conditional covering: Greedy heuristics and computational results," *Computers and Operations Research*, vol. 14, no. 1, pp. 11-18, 1987.
- [4] E. J. Cockayne, R. M. Dawes, and S. T. Hedetniemi, "Total domination in graphs," *Networks*, vol. 10, no. 3, pp. 211-219, 1980.
- [5] J. A. Horne and J. C. Smith, "Dynamic programming algorithms for the conditional covering problem on path and extended star graphs," *Networks*, vol. 46, no. 4, pp. 177-185, 2005.
- [6] J. A. Horne and J. C. Smith, "A dynamic programming algorithm for the conditional covering problem on tree graphs," *Networks*, vol. 46, no. 4, pp. 186-197, 2005.
- [7] R. Laskar, J. Pfaff, S. M. Hedetniemi, and S. T. Hedetniemi, "On the algorithmic complexity of total domination," *SIAM Journal on Algebraic and Discrete Methods*, vol. 5, no. 3, pp. 420-425, 1984.
- [8] C.-H. Lin and B.-F. Wang, "Efficient algorithms for the conditional covering problem on paths, extended stars, and trees," manuscript.
- [9] V. Lotfi and I. D. Moon, "Hybrid heuristics for conditional covering problems," *International Journal of Modelling and Simulation*, vol. 17, no. 3, pp. 185-190, 1997.
- [10] B. J. Lunday, J. C. Smith, and J. B. Goldberg, "Algorithms for solving the conditional

- covering problem on paths," *Naval Research Logistics*, vol. 52, no. 4, pp. 293-301, 2005.
- [11] I. D. Moon, "Conditional covering: Worst case analysis of greedy heuristics," *Journal of the Korean OR/MS Society*, vol. 15, no. 2, pp. 97-104, 1990.
 - [12] I. D. Moon and S. S. Chaudhry, "An analysis of network location problems with distance constraints," *Management Science*, vol. 30, no. 3, pp. 290-307, 1984.
 - [13] I. D. Moon and L. Papayanopoulos, "Facility location on a tree with maximum distance constraints," *Computers and Operations Research*, vol. 22, no. 9, pp. 905-914, 1995.
 - [14] J. Pfaff, R. Laskar, and S. T. Hedetniemi, "NP-completeness of total and connected domination, and irredundance for bipartite graphs," *Technical Report 428*, Department of Mathematical Sciences, Clemson University, Clemson, South Carolina, 1983.
 - [15] J. Pfaff, R. Laskar, and S. T. Hedetniemi, "Linear algorithms for independent domination and total domination in series-parallel graphs," *Congressus Numerantium* vol. 45, pp. 71-82, 1984.
 - [16] F. P. Preparata and M. I. Shamos, *Computational Geometry: an Introduction*, Springer-Verlag New York, Inc., New York, NY, 1985.
 - [17] D. Schilling, V. Jayaraman, and R. Barkhi, "A review of covering problems in facility location," *Location Science*, vol. 1, no. 1, pp. 25-55, 1993.
 - [18] C. Toregas, R. Swain, C. ReVelle, and L. Bergman, "The location of emergency service facilities," *Operations Research*, vol. 19, no. 6, pp. 1363-1373, 1971.

Appendix 1. A counterexample to Horne and Smith's path algorithm in [5]

Horne and Smith's algorithm is briefly described as follows. Their algorithm is based on the dynamic programming approach. Let $i \in V$ be a vertex. The *protected cost* $p(i)$ is the minimum cost to locate a facility at vertex i and cover vertices 1 through i , with no facilities placed at vertices $i + 1, i + 2, \dots, n$. The *unprotected cost* $u(i)$ is the minimum cost to locate a facility at vertex i and cover vertices 1 through $i - 1$, with no facilities placed at vertices $i + 1, i + 2, \dots, n$. Define the following subsets:

$GA_1(i) = \{k \mid k \in V, k < i \leq g(k), h(i) > k\}$, which contains the vertices $k < i$ that cover i , but are not covered by i ; and

$GB_1(i) = \{k \mid k \in V, k < i \leq g(k), h(i) \leq k\}$, which contains the vertices $k < i$ that cover i and are covered by i .

Assume that $p(i) \neq \infty$. Let $S \subseteq \{1, 2, \dots, i\}$ be a subset such that $i \in S$, S covers vertices 1 through i , and $c(S) = p(i)$. Let k be the largest vertex in $S - \{i\}$. Intuitively, i is covered by k and $S - \{i\}$ covers vertices 1 through $k - 1$. Therefore, Horne and Smith gave the following recurrence for $p(i)$:

$$p(i) = \min\left\{\min_{k \in GA_1(i)} p(k), \min_{k \in GB_1(i)} u(k)\right\} + c_i.$$

For brevity, their recurrence of $u(i)$ is omitted. In the following, we show that there is a flaw in the above recurrence. Consider the example in Figure A1. By definition, we have $p(1) = p(2) = \infty$, $u(1) = 1$, and $u(2) = \infty$. Consider the computation of $p(3)$. Since $GA_1(3) = \emptyset$ and $GB_1(3) = \{2\}$, according to the above recurrence, we have $p(3) = u(2) + c_3 = \infty$. However, since $\{2, 3\}$ covers vertices 1, 2, 3, the correct value of $p(3)$ is 2. Such a flaw comes from the improper intuition: $S - \{i\}$ covers vertices 1 through $k - 1$. Consider the example in Figure A1. The subset $\{2, 3\}$ covers vertices 1, 2, and 3, but vertex 2 does not cover vertex 1.

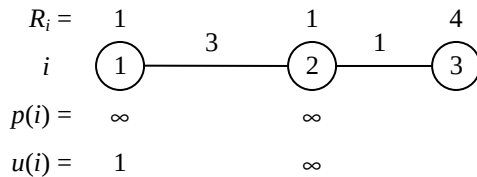


Figure A1. A counterexample, in which $c_1 = c_2 = c_3 = 1$.

Appendix 2. Proofs for Lemmas 7 and 12

Lemma 7. If k is not covered by i , $P'(i)$ is an optimal protected k -placement; otherwise, $P'(i)$ is an optimal unprotected k -placement.

Proof: We only prove the case that k is not covered by i . The proof for the other case is similar. Assume that k is not covered by i . By Lemma 6, $P'(i)$ covers 1, 2, ..., $k - 1$. Since k is not covered by i , $P'(i)$ also covers k ; otherwise $P(i)$ is not an i -cover. The vertex with the largest rank in $P'(i)$ is k . Thus, $P'(i)$ is a protected k -placement. Consider an optimal protected k -placement S . By Lemma 5, $\{i, k\}$ covers $k + 1, k + 2, \dots, i - 1$. If k also covers i , $S \cup \{i\}$ is a protected i -placement; otherwise, by Lemma 4 $\alpha(i) \in P(i)$ and $S \cup \{i, \alpha(i)\}$ is a protected i -placement. Therefore, by replacing $P'(i)$ with S , we can obtain from $P(i)$ another protected i -placement with cost $c(P(i)) - c(P'(i)) + c(S)$, from which we conclude that $P'(i)$ is optimal; otherwise $P(i)$ is also not optimal. Therefore, the lemma holds. $\square$

Lemma 12. The value of $c_{\beta(i)}$ can be computed in $O(n \log n)$ time for every $i \in V$.

Proof: By using a heap, Q , as a priority queue, all $c_{\beta(i)}$ can be computed as follows. First, by sorting, we construct a sequence $(w_1, w_2, \dots, w_{3n})$ that contains all $h(i)$, i , and $g(i)$, $i = 1, 2, \dots, n$. In the sequence, for each $i \in V$, all $h(k) \leq i$ appear before i and all $g(k) \geq i$ appear after i . Next, from left to right, we check each w_j and do the following: if w_j is some lower reach $h(i)$, insert c_i into Q ; if w_j is some upper reach $g(i)$, delete c_i from Q ; and if w_j is some vertex i , we first delete c_i from Q , then compute $c_{\beta(i)}$ as the minimum cost in Q , and then insert c_i back to Q . The time complexity is $O(n \log n)$. Thus, the lemma holds. $\square$